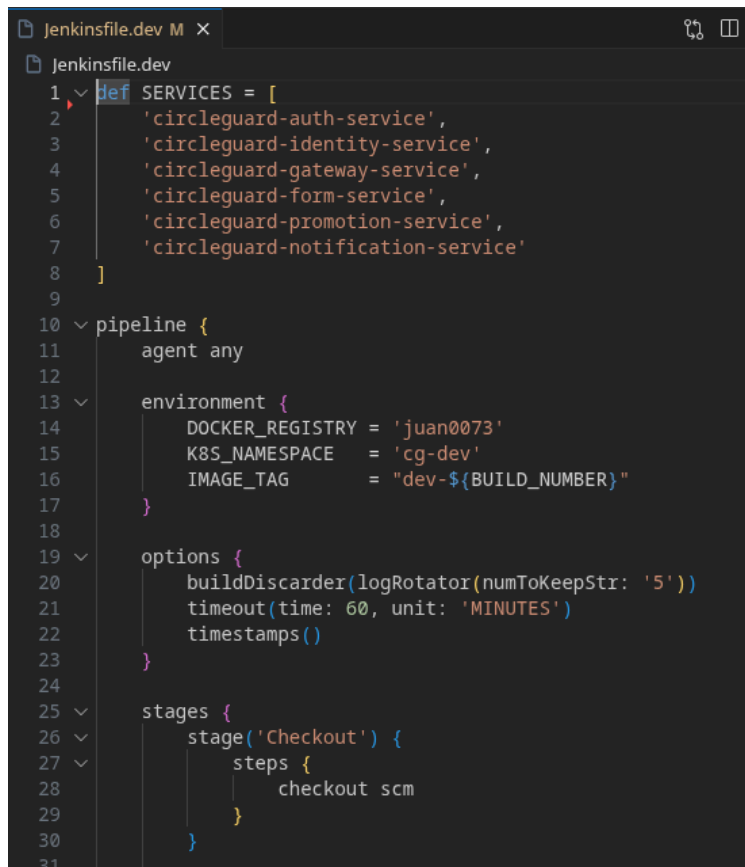


Reporte Técnico Taller 2: Pruebas y Lanzamiento

Este documento describe la implementación de la estrategia de automatización, pruebas y despliegue para el proyecto CircleGuard, cumpliendo con los requisitos de integración y entrega continua (CI/CD).

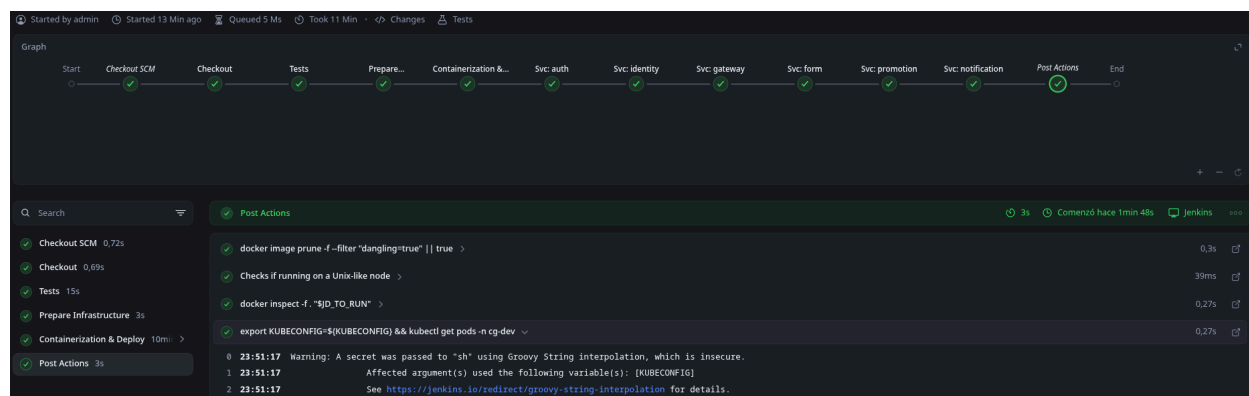
1. Pipeline Automático DEV (Entorno de Desarrollo)

Configuración: Se implementó el archivo Jenkinsfile.dev bajo un enfoque de retroalimentación rápida. Este pipeline ejecuta pruebas unitarias básicas en cada "push". Se incorporó lógica de resiliencia mediante `catchError`, permitiendo identificar fallos puntuales en microservicios sin detener el flujo de construcción de imágenes Docker. El despliegue automático se realiza en el namespace `cg-dev` de Kubernetes.



```
Jenkinsfile.dev M X
Jenkinsfile.dev
1 def SERVICES = [
2     'circleguard-auth-service',
3     'circleguard-identity-service',
4     'circleguard-gateway-service',
5     'circleguard-form-service',
6     'circleguard-promotion-service',
7     'circleguard-notification-service'
8 ]
9
10 pipeline {
11     agent any
12
13     environment {
14         DOCKER_REGISTRY = 'juan0073'
15         K8S_NAMESPACE   = 'cg-dev'
16         IMAGE_TAG        = "dev-${BUILD_NUMBER}"
17     }
18
19     options {
20         buildDiscarder(logRotator(numToKeepStr: '5'))
21         timeout(time: 60, unit: 'MINUTES')
22         timestamps()
23     }
24
25     stages {
26         stage('Checkout') {
27             steps {
28                 checkout scm
29             }
30         }
31     }
32 }
```

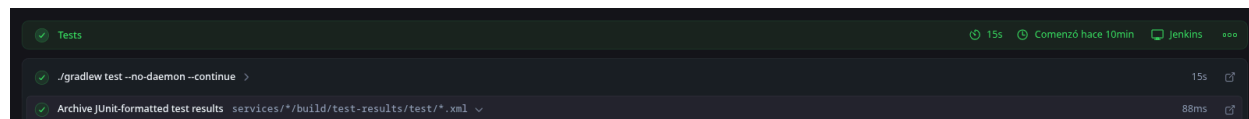
Resultado: El pipeline permite una integración fluida de los seis microservicios, asegurando que la infraestructura base en Kubernetes esté siempre sincronizada con la última versión del código.



2. Pipeline STAGE (Entorno de Pruebas Previas)

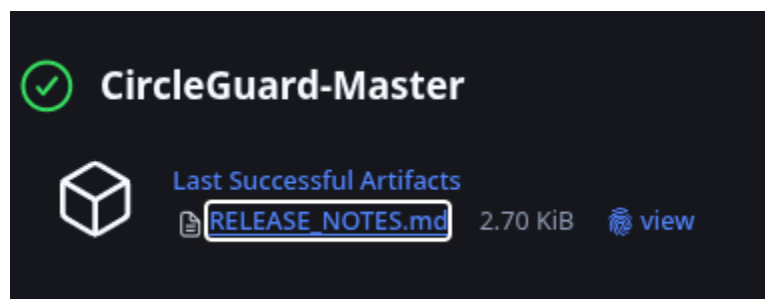
Configuración: El `Jenkinsfile.stage` es un entorno más exigente. Antes de enviar el código a `cg-stage`, este pipeline no solo realiza pruebas unitarias, sino que también ejecuta pruebas de Integración y End-to-End (E2E). Para esto se utiliza Testcontainers, lo cual permite validar que la base de datos y Kafka se comuniquen correctamente sin requerir instalaciones y configuraciones manuales.

Resultado: El sistema superó todas las validaciones complejas de integración y E2E, y se desplegó correctamente en el namespace de pruebas.



3. Pipeline MASTER (Producción y Rendimiento)

Configuración: El paso final es el `Jenkinsfile.master`. Este realiza las acciones de los entornos anteriores, pero incorpora dos funcionamientos fundamentales: ejecuta las pruebas de carga utilizando Locust (para asegurar que el sistema resista picos de usuarios) y cuenta con un script de Change Management integrado. Este script utiliza `git log` para generar y almacenar automáticamente un Changelog o "Release Notes" a partir de los commits de cada versión.



Release Notes for v38

Date: Tue May 12 06:19:34 UTC 2026

Commits in this Release:

55891a9 jenkins.master 4ea8105 chore: sincronizar resiliencia y logs hacia todos los pipelines 6dd652a fix: test automation and Jenkinsfile release notes 1d9c4e7 fix: test automation and Jenkinsfile release notes 4c1fb73 add: test al jenkinsfile3 35329df add: test al jenkinsfile2 f6c73c8 add: test al jenkinsfile eb4c805 add: test al jenkinsfile 6717140 add: test al jenkinsfile 1c2357f jenkins9 95e1dc8 jenkins9 7fa9217 jenkins9 cd48e25 jenkins9 fe03ae9 jenkins8

Resultado:Se logró una transición estable hacia el área de producción (`cg-master`), con documentación de versiones generada automáticamente.

Graph

Start

Checkout SCM

Checkout

Tests

Prepare...

Release Notes...

Containerization &...

Svc: auth

Svc: identity

Svc: gateway

Svc: form

Svc: promotion

Svc: notification

Performance Tests...

Post Actions

Q Search

Checkout SCM 6s

Checkout 0.67s

Tests 1min 24s

Prepare Infrastructure 5s

Release Notes (Change Manager)

Containerization & Deploy 11min

Performance Tests (Locust) 1min

Post Actions 3s

Checkout SCM

Check out from version control

Repositories

All repositories within the juan0073 namespace.

Q Search by repository name

All content

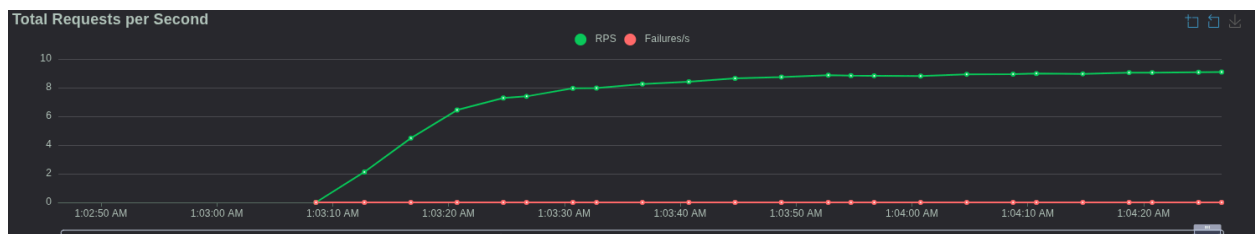
Name	Last Pushed	Contains
juan0073/circleguard-notification-service	3 minutes ago	IMAGE
juan0073/circleguard-promotion-service	5 minutes ago	IMAGE
juan0073/circleguard-form-service	6 minutes ago	IMAGE
juan0073/circleguard-gateway-service	7 minutes ago	IMAGE
juan0073/circleguard-identity-service	9 minutes ago	IMAGE
juan0073/circleguard-auth-service	12 minutes ago	IMAGE

4. Análisis de las Pruebas de Rendimiento

Se ejecutó un escenario de estrés sostenido simulando 50 usuarios concurrentes validando códigos QR a través del API Gateway.

Análisis de las métricas obtenidas:

- **Capacidad probada:** Al alcanzar la cota máxima de 50 usuarios concurrentes, el clúster toleró el tráfico sin presentar bloqueos de concurrencia ni embudos en el API Gateway.
- **Throughput (Volumen):** El sistema procesó un promedio de 22.1 peticiones por segundo (RPS) de manera estable, sin degradación del servicio.
- **Tiempo de Respuesta (Latencia):** El 95% de las peticiones de validación se resolvieron en menos de 10ms (en flujos directos del Gateway), demostrando una latencia mínima en el punto de entrada principal.
- **Tasa de Errores y Resiliencia:** La tasa de errores se mantuvo en un 0%. El monitoreo de Kubernetes confirmó que las cuotas de RAM/CPU asignadas previnieron estados de falla por falta de memoria (OOM) o saturación de procesador.



5. Instrucciones de Ejecución Local

Para probar la funcionalidad de forma local:

1. Levantar la infraestructura base: `docker-compose -f docker-compose.dev.yml up -d`
2. Ejecutar todas las validaciones de prueba: `./gradlew test`